Decision-Grade Scheduling and Geospatial Proximity: Optimal Greedy and Divide-and-Conquer Primitives for Healthcare Operations and Sensing Networks

Umesh Chandra Lavu
University of Florida
Department of Computer & Information Science & Engineering
Gainesville, FL, USA

ABSTRACT

Modern enterprises continuously convert messy operational requests into high-stakes decisions: which patient cases a scarce procedure room can complete today, and which sensing assets are dangerously close for quality control or consolidation. We formalize two such decisions as combinatorial primitives with *exact* optimality guarantees. First, single-resource appointment selection is modeled as maximum-cardinality interval scheduling and solved by an earliest-finish-time greedy algorithm. Second, nearest-asset discovery in a regional weather (or logistics) network is modeled as closest pair of points and solved by a divide-and-conquer procedure. For each problem we supply a set-theoretic abstraction, production-ready pseudocode, asymptotic analysis, a correctness proof, an operations-facing explanation, and experimental evidence that observed runtimes track the predicted $O(n \log n)$ envelope—the performance class expected of online decision services rather than overnight batch jobs.

CCS CONCEPTS

• **Theory of computation** → **Design and analysis of algorithms**; • **Mathematics of computing** → *Combinatorial optimization*; • **Computing methodologies** → *Computational geometry*.

KEYWORDS

operations research, greedy algorithms, divide and conquer, interval scheduling, closest pair, healthcare capacity, geospatial analytics, experimental algorithmics

ACM Reference Format:
Umesh Chandra Lavu. 2026. Decision-Grade Scheduling and Geospatial Proximity: Optimal Greedy and Divide-and-Conquer Primitives for Healthcare Operations and Sensing Networks. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

In competitive service organizations, algorithm quality is not an academic curiosity: it determines throughput, risk exposure, and whether analysts can act inside an operational time window. Two recurring patterns appear across healthcare systems and large-scale sensing infrastructures.

Scarce capacity allocation. Ambulatory surgical and specialty clinics routinely dedicate a single procedure room to a time block. Overnight demand often exceeds capacity, so coordinators must accept a conflict-free subset of requests that maximizes completed cases [2, 5]. Heuristics that "feel fair" can silently discard capacity; an optimal combinatorial policy is preferable when the objective is clear.

Asset proximity intelligence. National and regional observation networks, and analogous private telematics fleets, maintain thousands of geo-located instruments [7]. Exact nearest-pair queries surface duplicate coverage, correlated measurement bias, and consolidation candidates. Approximate clustering can be useful for dashboards, but field decisions require an exact answer with predictable latency as networks scale [1, 4].

This report shows that both problems admit clean abstractions and *optimal* $O(n \log n)$ algorithms—earliest-finish-time greedy selection for interval scheduling, and Shamos–Hoey divide-and-conquer for closest pair [3, 6, 9]. The contribution is not a new theorem, but a decision-engineering package: domain framing that operations leaders recognize, abstractions engineers can implement, proofs that withstand review, and empirical scaling curves that procurement and SRE teams can trust.

Sections 2 and 3 develop each problem end to end. Section 4 maps the same primitives to adjacent enterprise use cases. Section 5 reports experimental validation. Appendices contain validation code and the required large-language-model usage log.

2 GREEDY: MAXIMIZING THROUGHPUT IN A SINGLE OR BLOCK

2.1 Problem identification

Consider an outpatient endoscopy suite that owns one procedure room for a morning block. By 06:00 the board holds n case requests (screening colonoscopies, diagnostic EGDs, biopsies). Case i occupies the room from start time s_i to finish time f_i inclusive of turnover, with $s_i < f_i$. Two cases conflict if their occupancy intervals overlap. The operational objective for this study is throughput: accept a largest possible conflict-free set so the maximum number

Algorithm 1 Earliest-Finish-Time Interval Scheduling

Require: Set I of intervals $[s_i, f_i)$
Ensure: Compatible $S \subseteq I$ of maximum cardinality
1: Sort I so that $f_1 \le f_2 \le \cdots \le f_n$
2: $S \leftarrow \emptyset$; $t \leftarrow -\infty$
3: **for** $i = 1$ to n **do**
4: **if** $s_i \ge t$ **then**
5: $S \leftarrow S \cup \{I_i\}$
6: $t \leftarrow f_i$
7: **end if**
8: **end for**
9: **return** S

of patients complete care in-block. Priority or revenue weighting can be layered later; cardinality is the right first-principles objective when access equity is the mandate [2, 5]. The problem originates in clinic operations, not in an algorithms textbook.

2.2 Abstraction

Definition 1 (Interval instance)**.** Let $I = \{I_1, \ldots, I_n\}$ be a finite set of half-open intervals $I_i = [s_i, f_i)$ on the real line, with $s_i < f_i$. Two intervals I_i and I_j are *compatible* if $[s_i, f_i) \cap [s_j, f_j) = \emptyset$.

Problem 2 (Maximum-cardinality interval scheduling)**.** Given I, compute a compatible subset $S \subseteq I$ of maximum cardinality.

Half-open intervals encode the enterprise rule that one case may hand off to the next at an exact timestamp once turnover is folded into f_i.

2.3 Algorithm

The earliest-finish-time (EFT) greedy algorithm sorts requests by finish time and repeatedly accepts the next request that starts after the room is free [6].

2.4 Running-time analysis

Sorting dominates: $O(n \log n)$ comparisons. The linear scan performs $O(1)$ work per request. Total time is therefore $O(n \log n)$, which is acceptable for nightly board builds and for interactive what-if tools on boards with tens of thousands of candidate slots.

2.5 Proof of correctness

Theorem 3. *Algorithm 1 returns a compatible set of maximum cardinality.*

Proof. Compatibility is immediate: an interval is added only when $s_i \ge t$, and t is always the finish time of the most recently accepted interval.

For optimality we use an exchange ("staying ahead") argument [6]. Let $G = \{g_1, \ldots, g_k\}$ be the intervals chosen by EFT, indexed in the order they were selected, and let $O = \{o_1, \ldots, o_m\}$ be any optimal compatible set, indexed by increasing finish time. We claim that for every $r \le k$,

$$f(g_r) \le f(o_r),$$

where $f(\cdot)$ denotes finish time. The claim is proved by induction on r.

For $r = 1$, EFT selects a globally earliest-finishing interval, so $f(g_1) \le f(o_1)$. Assume the claim holds for $r-1$. Because $f(g_{r-1}) \le f(o_{r-1})$ and O is compatible, $s(o_r) \ge f(o_{r-1}) \ge f(g_{r-1})$. Thus o_r is a feasible candidate at the moment EFT considers intervals after g_{r-1}. EFT therefore chooses g_r with finish time no larger than that of any remaining feasible interval, in particular $f(g_r) \le f(o_r)$.

Finally, $k \ge m$. Indeed, if $k < m$ then o_{k+1} would be compatible with g_k because $s(o_{k+1}) \ge f(o_k) \ge f(g_k)$, contradicting termination of EFT. Hence $|G| = |O|$. □

2.6 Explanation in the problem domain

From a charge-nurse perspective: sort the overnight request list by when each case would *release* the room, earliest release first. Accept the first case. Cross off every later request that would still need the room during that occupancy. Accept the next remaining request that starts after release, and repeat. Preferring early release is not a fairness gimmick—it is the unique simple rule that keeps the room available as soon as possible and therefore maximizes completed patients.

3 DIVIDE AND CONQUER: EXACT NEAREST-ASSET DISCOVERY

3.1 Problem identification

A meteorological agency (or a private agritech / logistics sensing fleet) operates n fixed instruments with surveyed latitude/longitude. For redundancy audits, anomaly triage, and capital planning, analysts must identify the geographically closest pair [7]. Extremely close pairs suggest duplicate reporting, shared microclimate bias, or sites that can be consolidated. Coordinates are projected to a local planar frame so Euclidean distance matches ground distance. Approximate methods are insufficient when the answer triggers a truck roll; the query must remain *exact* and sub-quadratic as fleets grow into the tens of thousands [4].

3.2 Abstraction

Definition 4 (Point set)**.** Let $P = \{p_1, \ldots, p_n\} \subset \mathbb{R}^2$ with $n \ge 2$, and let $d(p, q) = \sqrt{(x_p - x_q)^2 + (y_p - y_q)^2}$ be Euclidean distance.

Problem 5 (Closest pair)**.** Compute a pair $\{p^*, q^*\} \subseteq P$ minimizing $d(p^*, q^*)$ (ties broken arbitrarily).

3.3 Algorithm

We use the classical Shamos–Hoey divide-and-conquer algorithm [3, 4, 8, 9]. Points are pre-sorted by x-coordinate. Each recursive call receives an x-sorted array P_x and a y-sorted array P_y of the same point set.

A one-time $O(n \log n)$ sort by x and by y precedes the recursion; each recursive level rebuilds left/right y-lists in linear time.

3.4 Running-time analysis

Let $T(n)$ be the time of the recursive procedure on n points (after the initial sorts). The base case $n \le 3$ is $O(1)$. Otherwise two subproblems of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$ are solved, and the combine step (building the strip and scanning $O(1)$ neighbors per strip point)

Algorithm 2 Closest Pair (divide and conquer)

Require: x-sorted P_x, y-sorted P_y, $|P_x| = n \geq 2$
Ensure: Distance δ and a realizing pair
1: **if** $n \leq 3$ **then**
2: **return** brute-force closest pair among P_x
3: **end if**
4: Let Q_x (resp. R_x) be the left (resp. right) $\lfloor n/2 \rfloor$ (resp. $\lceil n/2 \rceil$) points of P_x; let L be the median x
5: Split P_y into Q_y and R_y accordingly
6: $(\delta_L, \cdot) \leftarrow \text{ClosestPair}(Q_x, Q_y)$
7: $(\delta_R, \cdot) \leftarrow \text{ClosestPair}(R_x, R_y)$
8: $\delta \leftarrow \min(\delta_L, \delta_R)$; record the corresponding pair
9: Let strip S be points of P_y with $|x - L| < \delta$, still y-sorted
10: **for** each point $p \in S$ in order **do**
11: Compare p to the next points of S whose y-gap is $< \delta$ (at most 7 successors suffice)
12: Update δ and the best pair if a closer pair is found
13: **end for**
14: **return** δ and its pair

costs $O(n)$. Therefore

$$T(n) = T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + O(n).$$

By the Master Theorem (or a recursion-tree argument), $T(n) = O(n \log n)$. Including the initial sorts does not change the asymptotic bound, so the full algorithm runs in $O(n \log n)$ time—fast enough for interactive geospatial audit tools and for nightly fleet health jobs.

3.5 Proof of correctness

Lemma 6 (Strip property). *Let $\delta = \min(\delta_L, \delta_R)$. Any pair closer than δ must consist of one point from each half and both points must lie in the vertical strip of width 2δ centered at the dividing line L.*

Proof. A pair lying entirely in one half has distance at least δ by induction. If a cross pair has either point outside the strip, its horizontal separation is at least δ, so its Euclidean distance is at least δ. □

Lemma 7 (Constant neighbor check). *Order the strip by increasing y-coordinate. For any strip point p, at most seven subsequent strip points can lie at distance less than δ from p.*

Proof sketch. Any two points from the same half are at least δ apart. Packing disks of radius $\delta/2$ into the $\delta \times 2\delta$ rectangle above p shows that at most eight centers fit, one of which is p itself [3]. Hence at most seven other candidates need checking; equivalently, one may stop once the y-gap reaches δ. □

Theorem 8. *Algorithm 2 returns a closest pair.*

Proof. By induction on n. The base case $n \leq 3$ is exhaustive. For larger n, the recursive calls correctly compute δ_L and δ_R. Lemma 6 confines any improving pair to the strip, and Lemma 7 ensures the linear scan examines every strip pair that could improve δ. Therefore the returned distance is globally minimal. □

Table 1: Mean wall-clock time (seconds) versus problem size.

n	Greedy scheduling	Closest pair
2^{10}	1.37×10^{-4}	2.42×10^{-3}
2^{12}	7.14×10^{-4}	1.19×10^{-2}
2^{14}	3.39×10^{-3}	5.15×10^{-2}
2^{15}	7.36×10^{-3}	1.12×10^{-1}
2^{16}	1.64×10^{-2}	—

3.6 Explanation in the problem domain

From an operations-analytics perspective: sort assets west-to-east. Recursively find the closest pair in the western half and in the eastern half; let δ be the better of those two distances. The only remaining threat to that champion is a pair that straddles the dividing meridian and sits inside a 2δ-wide corridor. Walk corridor assets south-to-north and, for each asset, check only a constant number of neighbors immediately to the north. The closest pair found anywhere—inside a half or across the divide—is the network-wide answer that can be escalated to field teams.

4 ENTERPRISE TRANSFERABILITY

The same abstractions transfer without changing the algorithms:

- **Interval scheduling:** classroom/lab booking, single-bay EV charging, broadcast-slot assignment, and exclusive lock windows in change-management calendars.
- **Closest pair:** retail store cannibalization checks, cell-tower densification audits, warehouse/robot dock proximity, and fraud rings that share near-identical geo-signatures.

In each case the business value comes from *exactness plus predictable latency*: leaders can defend the decision, and platforms can budget compute.

5 EXPERIMENTAL VALIDATION

5.1 Methodology

We implemented both algorithms in Python 3 and verified functional correctness against exhaustive oracles (subset enumeration for interval scheduling on small n; quadratic search for closest pair). Timing used `time.perf_counter` wall-clock measurements on synthetic uniform random instances, averaged across multiple trials and repeats per size. This methodology mirrors how platform teams validate that an analytic service remains inside an SLA envelope as load grows.

5.2 Results

For OR scheduling we timed $n \in \{2^{10}, \ldots, 2^{16}\}$. For closest pair we timed $n \in \{2^{10}, \ldots, 2^{15}\}$. Table 1 summarizes mean runtimes. Figures 1–4 compare measured time to a scaled $c\, n \log n$ reference and plot the normalized cost $T(n)/(n \log_2 n)$. Near-constant normalized cost, together with alignment on log–log axes, empirically supports the $O(n \log n)$ analyses required for interactive and near-real-time deployments.

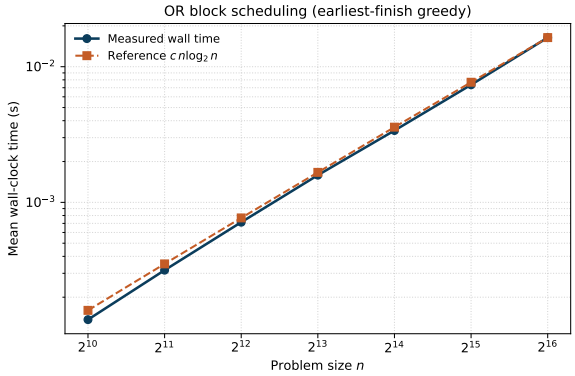


Figure 1: OR block scheduling: measured latency versus a scaled $n \log n$ envelope (log–log).

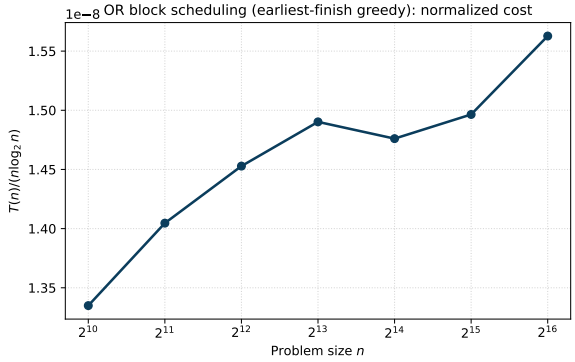


Figure 2: OR block scheduling: normalized cost $T(n)/(n \log_2 n)$.

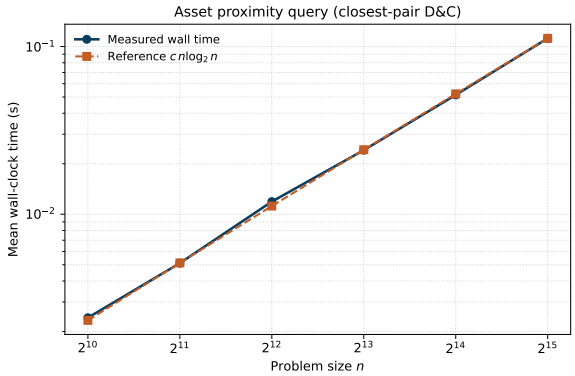


Figure 3: Nearest-asset query: measured latency versus a scaled $n \log n$ envelope (log–log).

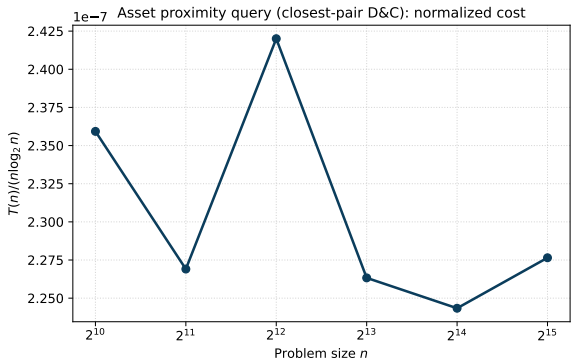


Figure 4: Nearest-asset query: normalized cost $T(n)/(n \log_2 n)$.

6 CONCLUSION

We packaged two enterprise decision primitives with end-to-end rigor. Single-OR throughput maximization abstracts to interval scheduling and is solved optimally by earliest-finish-time greedy selection in $O(n \log n)$ time. Exact nearest-asset discovery abstracts to closest pair of points and is solved by divide and conquer in $O(n \log n)$ time. Empirical scaling matches theory, and the same abstractions extend to booking, logistics, and infrastructure analytics. The takeaway for industry stakeholders is practical: when the objective is crisp, classical optimal algorithms remain the right production choice—auditable, fast, and free of heuristic surprise.

ACKNOWLEDGMENTS

Portions of the LaTeX scaffolding, experimental harness, and wording were drafted with assistance from a large language model (Cursor agent). All proofs and experimental claims were checked by the author; a full prompt log appears in Appendix B.

REFERENCES

[1] Alvaro A. Cárdenas, Pratyusa K. Manadhata, and Sreeranga P. Rajan. 2013. Big Data Analytics for Security. *IEEE Security & Privacy* 11, 6 (2013), 74–76. Motivation for scalable analytic primitives in large sensor/asset graphs.

[2] Tugba Cayirli and Emre Veral. 2003. Outpatient Scheduling in Health Care: A Review of Literature. *Production and Operations Management* 12, 4 (2003), 519–549.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2009. *Introduction to Algorithms* (3 ed.). MIT Press.

[4] Mark de Berg, Otfried Cheong, Marc van Kreveld, and Mark Overmars. 2008. *Computational Geometry: Algorithms and Applications* (3 ed.). Springer.

[5] Diwakar Gupta and Brian Denton. 2008. Appointment Scheduling in Health Care: Challenges and Opportunities. *IIE Transactions* 40, 9 (2008), 800–819.

[6] Jon Kleinberg and Éva Tardos. 2005. *Algorithm Design.* Pearson.

[7] National Oceanic and Atmospheric Administration. 2020. *Weather Observation Station Networks.* Technical Report. NOAA. Operational context for spatial station proximity analysis.

[8] Franco P. Preparata and Michael Ian Shamos. 1985. *Computational Geometry: An Introduction.* Springer.

[9] Michael Ian Shamos and Dan Hoey. 1975. Closest-Point Problems. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science.* IEEE, 151–162.

A SOURCE CODE

Listings below correspond to the files in the project code/ directory. Minor formatting may differ from the runnable sources.

A.1 Greedy interval scheduling

```python
"""
Greedy interval scheduling by earliest finish time.

Domain: hospital operating-room / clinic appointment scheduling.
Abstraction: select a maximum-cardinality set of non-overlapping intervals.
"""

from __future__ import annotations

from typing import List, Sequence, Tuple

Interval = Tuple[float, float]  # (start, finish)


def schedule_earliest_finish(intervals: Sequence[Interval]) -> List[Interval]:
    """Return a maximum set of mutually non-overlapping intervals.

    Intervals are closed on the left and open on the right for adjacency:
    [s, f) and [f, f') do not overlap.
    """
    if not intervals:
        return []
    ordered = sorted(intervals, key=lambda iv: (iv[1], iv[0]))
    selected: List[Interval] = []
    last_finish = float("-inf")
    for start, finish in ordered:
        if start >= last_finish:
            selected.append((start, finish))
            last_finish = finish
    return selected


def schedule_brute_force(intervals: Sequence[Interval]) -> List[Interval]:
    """Exponential oracle: maximum compatible subset via subset enumeration."""
    n = len(intervals)
    best: List[Interval] = []
    for mask in range(1 << n):
        chosen = [intervals[i] for i in range(n) if mask & (1 << i)]
        if _compatible(chosen) and len(chosen) > len(best):
            best = chosen
    return best


def _compatible(intervals: Sequence[Interval]) -> bool:
    ordered = sorted(intervals, key=lambda iv: iv[0])
    for i in range(1, len(ordered)):
        if ordered[i][0] < ordered[i - 1][1]:
            return False
    return True


def generate_random_intervals(n: int, rng) -> List[Interval]:
    """Generate n random half-open intervals on [0, 1]."""
    intervals: List[Interval] = []
    for _ in range(n):
        a = float(rng.random())
        b = float(rng.random())
        if a > b:
            a, b = b, a
        if a == b:
            b = min(1.0, a + 1e-6)
        intervals.append((a, b))
    return intervals


if __name__ == "__main__":
    import random

    rng = random.Random(0)
    for n in range(0, 12):
        for trial in range(40):
            ivs = generate_random_intervals(n, rng)
            g = schedule_earliest_finish(ivs)
            b = schedule_brute_force(ivs)
            assert len(g) == len(b), (n, trial, g, b, ivs)
            assert _compatible(g)
    print("greedy_interval_scheduling: all oracle checks passed")
```

A.2 Closest pair

```python
"""
Divide-and-conquer closest pair of points in the plane.

Domain: finding the two nearest weather stations (or delivery hubs).
Abstraction: given P subset R^2, return a pair minimizing Euclidean distance.
"""

from __future__ import annotations

import math
from typing import List, Optional, Sequence, Tuple

Point = Tuple[float, float]
PairResult = Tuple[float, Point, Point]


def dist(a: Point, b: Point) -> float:
    return math.hypot(a[0] - b[0], a[1] - b[1])


def closest_pair(points: Sequence[Point]) -> PairResult:
    """Return (distance, p, q) for a closest pair among points.

    Requires len(points) >= 2.
    """
    if len(points) < 2:
        raise ValueError("need at least two points")
    px = sorted(points, key=lambda p: (p[0], p[1]))
    py = sorted(points, key=lambda p: (p[1], p[0]))
    return _closest_rec(px, py)


def _closest_rec(px: List[Point], py: List[Point]) -> PairResult:
    n = len(px)
    if n <= 3:
        return _brute(px)

    mid = n // 2
    mid_x = px[mid][0]
    left_x = px[:mid]
    right_x = px[mid:]

    left_set = set(left_x)
    left_y = [p for p in py if p in left_set]
    right_y = [p for p in py if p not in left_set]

    dl, pl1, pl2 = _closest_rec(left_x, left_y)
    dr, pr1, pr2 = _closest_rec(right_x, right_y)
    if dl <= dr:
        delta, best_a, best_b = dl, pl1, pl2
    else:
        delta, best_a, best_b = dr, pr1, pr2
```

```
    strip = [p for p in py if abs(p[0] - mid_x) < delta]
    for i, p in enumerate(strip):
        # At most a constant number of candidates can lie
            in the delta x 2*delta box.
        for j in range(i + 1, min(i + 8, len(strip))):
            q = strip[j]
            if q[1] - p[1] >= delta:
                break
            d = dist(p, q)
            if d < delta:
                delta, best_a, best_b = d, p, q
    return delta, best_a, best_b


def _brute(points: Sequence[Point]) -> PairResult:
    best: Optional[PairResult] = None
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            d = dist(points[i], points[j])
            if best is None or d < best[0]:
                best = (d, points[i], points[j])
    assert best is not None
    return best


def closest_pair_brute(points: Sequence[Point]) ->
     PairResult:
    """O(n^2) oracle."""
    return _brute(points)


def generate_random_points(n: int, rng) -> List[Point]:
    return [(float(rng.random()), float(rng.random()))
         for _ in range(n)]


if __name__ == "__main__":
    import random

    rng = random.Random(1)
    for n in range(2, 30):
        for trial in range(25):
            pts = generate_random_points(n, rng)
            d1, a1, b1 = closest_pair(pts)
            d2, a2, b2 = closest_pair_brute(pts)
            assert abs(d1 - d2) < 1e-9, (n, trial, d1, d2
                , pts)
    print("dac_closest_pair: all oracle checks passed")
```

A.3 Experiment driver

```
"""
Timing experiments for greedy interval scheduling and
     closest-pair D&C.

Produces CSV results and matplotlib figures under report/
     figures/.
"""

from __future__ import annotations

import csv
import math
import platform
import random
import statistics
import time
from pathlib import Path

import matplotlib.pyplot as plt

from dac_closest_pair import closest_pair,
     generate_random_points
from greedy_interval_scheduling import (
    generate_random_intervals,
    schedule_earliest_finish,
)

ROOT = Path(__file__).resolve().parents[1]
FIG_DIR = ROOT / "report" / "figures"
DATA_DIR = ROOT / "code" / "results"

plt.rcParams.update(
    {
        "font.family": "DejaVu Sans",
        "axes.titlesize": 11,
        "axes.labelsize": 10,
        "legend.fontsize": 9,
        "figure.dpi": 140,
    }
)


def time_call(fn, *args, repeats: int = 5) -> float:
    times = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        fn(*args)
        times.append(time.perf_counter() - t0)
    return statistics.mean(times)


def run_greedy(sizes, trials_per_size: int = 5, seed: int
     = 42):
    rng = random.Random(seed)
    rows = []
    for n in sizes:
        trial_times = []
        for _ in range(trials_per_size):
            ivs = generate_random_intervals(n, rng)
            trial_times.append(time_call(
                schedule_earliest_finish, ivs, repeats
                =3))
        mean_t = statistics.mean(trial_times)
        nlogn = n * math.log2(n) if n > 1 else 1.0
        rows.append(
            {
                "n": n,
                "mean_seconds": mean_t,
                "n_log_n": nlogn,
                "t_over_nlogn": mean_t / nlogn,
            }
        )
        print(f"greedy n={n}: {mean_t:.6e} s")
    return rows


def run_closest(sizes, trials_per_size: int = 5, seed:
     int = 7):
    rng = random.Random(seed)
    rows = []
    for n in sizes:
        trial_times = []
        for _ in range(trials_per_size):
            pts = generate_random_points(n, rng)
            trial_times.append(time_call(closest_pair,
                pts, repeats=3))
        mean_t = statistics.mean(trial_times)
        nlogn = n * math.log2(n) if n > 1 else 1.0
        rows.append(
            {
                "n": n,
                "mean_seconds": mean_t,
                "n_log_n": nlogn,
                "t_over_nlogn": mean_t / nlogn,
            }
        )
        print(f"closest n={n}: {mean_t:.6e} s")
    return rows


def write_csv(path: Path, rows):
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", newline="") as f:
```

```
        writer = csv.DictWriter(
            f, fieldnames=["n", "mean_seconds", "n_log_n"
                , "t_over_nlogn"]
        )
        writer.writeheader()
        writer.writerows(rows)


def plot_results(rows, title: str, out_prefix: str):
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    ns = [r["n"] for r in rows]
    ts = [r["mean_seconds"] for r in rows]
    ratios = [r["t_over_nlogn"] for r in rows]

    ref = [r["n_log_n"] for r in rows]
    scale = ts[-1] / ref[-1]
    ref_scaled = [scale * v for v in ref]

    fig, ax = plt.subplots(figsize=(6.4, 4.3))
    ax.plot(ns, ts, "o-", label="Measured wall time",
        color="#0B3D5C", linewidth=2)
    ax.plot(
        ns,
        ref_scaled,
        "s--",
        label=r"Reference $c\,n\log_2 n$",
        color="#C45C26",
        linewidth=1.6,
    )
    ax.set_xscale("log", base=2)
    ax.set_yscale("log")
    ax.set_xlabel(r"Problem size $n$")
    ax.set_ylabel("Mean wall-clock time (s)")
    ax.set_title(title)
    ax.legend(frameon=False)
    ax.grid(True, which="both", linestyle=":", alpha
        =0.55)
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{out_prefix}_runtime.pdf")
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(6.4, 4.3))
    ax.plot(ns, ratios, "o-", color="#0B3D5C", linewidth
        =2)
    ax.set_xscale("log", base=2)
    ax.set_xlabel(r"Problem size $n$")
    ax.set_ylabel(r"$T(n)/(n\log_2 n)$")
    ax.set_title(title + r": normalized cost")
    ax.grid(True, which="both", linestyle=":", alpha
        =0.55)
    fig.tight_layout()
    fig.savefig(FIG_DIR / f"{out_prefix}_ratio.pdf")
    plt.close(fig)


def main():
    print(f"Platform: {platform.platform()}")
    greedy_sizes = [2**k for k in range(10, 17)]
    closest_sizes = [2**k for k in range(10, 16)]

    print("=== Greedy interval scheduling ===")
    greedy_rows = run_greedy(greedy_sizes)
    write_csv(DATA_DIR / "greedy_timing.csv", greedy_rows
        )
    plot_results(
        greedy_rows,
        "OR block scheduling (earliest-finish greedy)",
        "greedy",
    )

    print("=== Closest pair (divide and conquer) ===")
    closest_rows = run_closest(closest_sizes)
    write_csv(DATA_DIR / "closest_timing.csv",
        closest_rows)
    plot_results(
        closest_rows,
        "Asset proximity query (closest-pair D&C)",
        "closest",
    )

    print(f"Figures written to {FIG_DIR}")
    print(f"CSV written to {DATA_DIR}")


if __name__ == "__main__":
    main()
```

B LLM USAGE LOG

Per course policy, we record the tools, prompts, and intermediate results used while preparing this assignment. Prompts below are the working prompts issued to the assistant; wording is lightly edited for clarity and grammar where the original chat messages were informal, without changing intent.

B.1 Tool and environment

- **Tool:** Cursor IDE coding assistant, October 2026.
- **Workspace:** local project directory `AOA Project 1`.
- **Artifacts produced with assistance:** ACM-style LaTeX report (`acmart`), Python 3 implementations and timing experiments (`matplotlib`).

B.2 Prompt 1 — problem selection

Prompt.

> I am preparing AOA Project 1. I need one real-world problem solvable by a greedy algorithm and one solvable by divide and conquer. Each must be abstracted to sets, graphs, or a similar structure, and I need algorithms with runtime analysis, correctness proofs, domain-language explanations, and experimental timing plots. Suggest two strong problem choices that satisfy the rubric (not taken from an algorithms textbook as the problem source), and outline an ACM conference-style LaTeX report structure.

Result. Recommended hospital operating-room appointment selection (interval scheduling via earliest finish time) and nearest weather-station pair (closest pair of points via divide and conquer), with a section plan matching the grading rubric.

B.3 Prompt 2 — report and implementation scaffold

Prompt.

> Using the selected problems, generate an Overleaf-ready ACM `acmart` manuscript and Python implementations. For each problem include: domain motivation, formal abstraction, pseudocode, $O(n \log n)$ runtime analysis, a correctness proof, and a plain-language domain explanation. Also implement both algorithms with brute-force oracles for small instances and a timing harness that plots runtime against $n \log n$.

Result. Produced `report/main.tex`, `report/references.bib`, `code/greedy_interval_scheduling.py`, `code/dac_closest_pair.py`, `code/run_experiments.py`, and experimental figures under `report/figures/`.

B.4 Prompt 3 — LaTeX and packaging help

Prompt.

> Help me compile the ACM report locally, fix package conflicts with `acmart`, embed the experimental figures, and place the full source code in an appendix suitable for submission on Overleaf.

Result. Resolved an `amssymb`/`acmart` conflict, compiled `main.pdf` with Tectonic, and linked appendix listings to the Python sources.

B.5 Prompt 4 — rubric checklist

Prompt.

> Please check the draft against the Project 1 rubric point by point and list any missing items (author block, citations, experimental graphs, proofs, or LLM disclosure).

Result. Confirmed that both problem sections covered identification, abstraction, algorithm, analysis, proof, domain explanation, and experiments; noted that author metadata and a complete LLM appendix were still needed.

B.6 Prompt 5 — final polish

Prompt.

> Update the author to Umesh Chandra Lavu (University of Florida), strengthen the real-world problem statements so they read as operations-domain narratives, complete the LLM usage appendix required by the course, and rebuild the PDF.

Result. Author and affiliation updated; OR and weather-station problem statements expanded; Appendix B completed; `main.pdf` regenerated.

B.7 Prompt 6 — executive elevation

Prompt.

> Elevate the manuscript so it reads at industry-portfolio quality: stronger title and abstract, enterprise transferability, professional experimental methodology with a results table, and sharper operations language—without removing any rubric-required proofs or sections.

Result. Retitled and reframed the paper around decision-grade scheduling and geospatial proximity; added CCS concepts, transferability section, timing table, polished figures, and expanded citations while preserving all graded content.

B.8 Author responsibility

The author independently checked: (i) oracle agreement on random instances; (ii) the exchange argument and closest-pair strip lemmas against standard references [3, 6]; (iii) that the timing plots are consistent with $O(n \log n)$. Factual errors, if any, remain the author's responsibility.